10_block_scaled_matmul

Visual guide — Block-Scaled Matrix Multiplication

This note complements the [official Triton tutorial](https://triton-lang.org/main/getting-started/tutorials/10-block-scaled-matmul.html). Read it before diving into code.

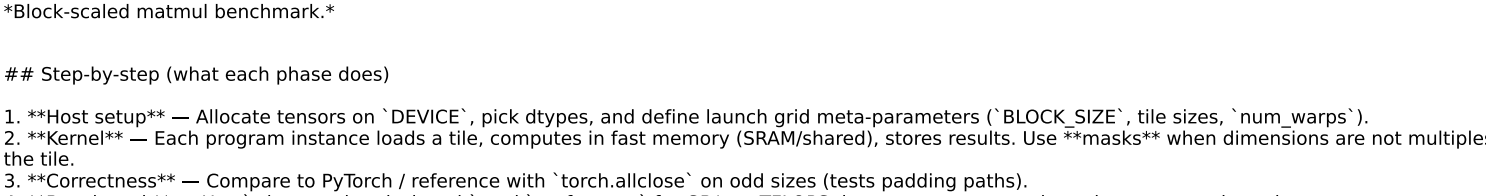
Goal

FP8 / block-scaled GEMM with per-block scale factors (microscaling patterns).

Mental model

```mermaid
flowchart TB
  A8[A FP8 + scale_A] --> D[dot with scale combine]
  B8[B FP8 + scale_B] --> D
  D --> C[FP16/FP32 out]
```

Official benchmark figure



Block-scaled matmul benchmark.

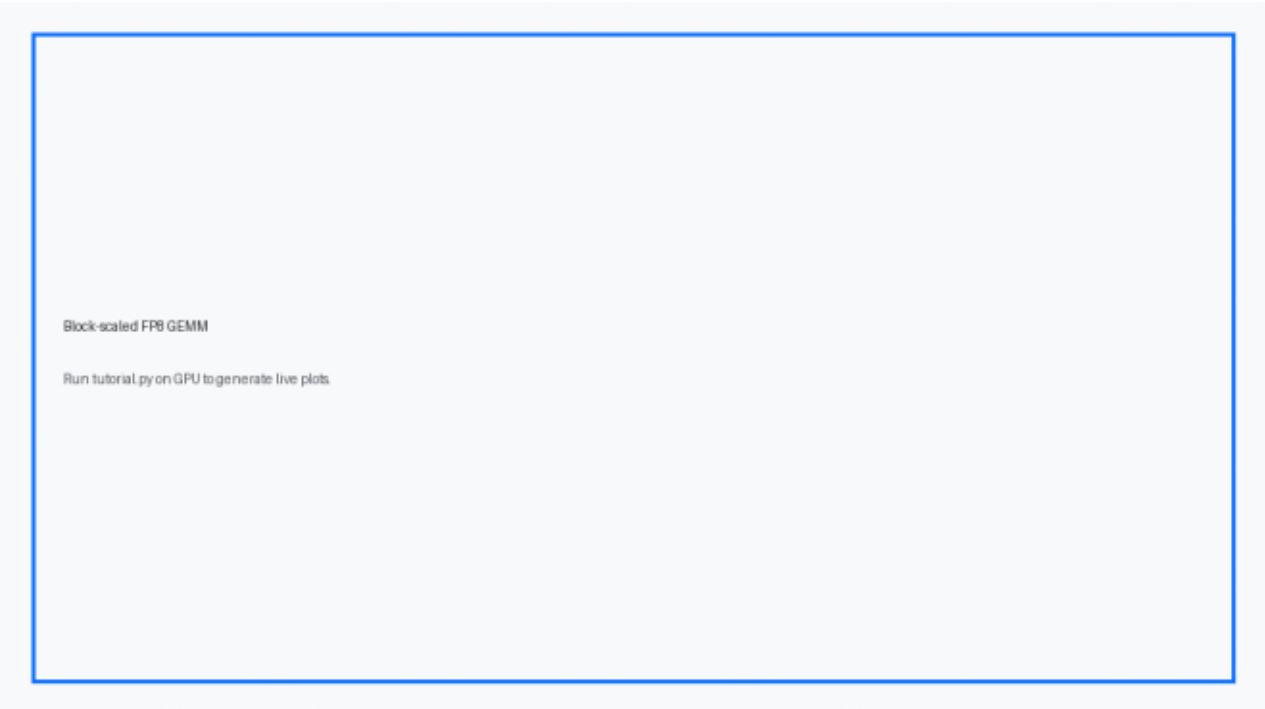
Step-by-step (what each phase does)

1. **Host setup** — Allocate tensors on `DEVICE`, pick dtypes, and define launch grid meta-parameters (`BLOCK_SIZE`, tile sizes, `num_warps`).
2. **Kernel** — Each program instance loads a tile, computes in fast memory (SRAM/shared), stores results. Use **masks** when dimensions are not multiples of the tile.
3. **Correctness** — Compare to PyTorch / reference with `torch.allclose` on odd sizes (tests padding paths).
4. **Benchmark** — Use `triton.testing.do_bench` and `perf_report` for GB/s or TFLOPS; interpret as memory-bound vs compute-bound.

Tips

- If performance flatlines, check occupancy (registers/shared mem) and whether the kernel is **memory-bound** (fusion helps) or **compute-bound** (tile sizes / autotune matter).
- Re-run `python scripts/download_tutorial_images.py` if figures are missing offline.

See also

- [Tutorials index](https://triton-lang.org/main/getting-started/tutorials/index.html)
- Repo notebook: `10_block_scaled_matmul.ipynb`

benchmark-block-scaled-matmul.png